Diseño de Topologías y Casos de Uso Empresariales

Capítulo 1 · Fundamentos de Infraestructura para LLMs · Sesión 2 (180 min)

Inferencia online y por lotes · Multi-región y redundancia · Selección de servicios · Diseño preliminar

Pregunta guía: ¿la misma topología sirve para todo?

- El asistente universitario de la Sesión 1 ahora tiene tres cargas de trabajo distintas:
  - Estudiantes conversando en temporada de matrículas: alguien espera cada respuesta
  - 20.000 PQRS al mes que se clasifican cada noche: nadie espera, importa el plazo
  - Reindexación semanal de documentos para RAG: volumen alto, sin urgencia
- Nuevo requisito institucional: el asistente debe seguir operando si cae una región de la nube
- Las cifras son hipótesis de diseño, no mediciones

Online · conversación
SLO de latencia

Batch · PQRS nocturnas
SLO de plazo

Batch · reindexación
SLO de plazo y costo

¿Una topología, dos o tres?

Resultados de aprendizaje y agenda

RA	Al finalizar podrá…	Bloque
RA1	Distinguir inferencia online, asíncrona y por lotes por su SLO, costo y patrón de arquitectura	Unidad A · Demo 1
RA2	Calcular la disponibilidad compuesta de una topología y relacionarla con RTO y RPO	Unidad B
RA3	Diseñar topologías redundantes (multizona, activo-pasivo, activo-activo) con patrones de resiliencia	Unidad B · Demo 2
RA4	Seleccionar servicios gestionados de Azure, Google Cloud y AWS según el caso de uso	Unidad C · Demo 3
RA5	Evaluar residencia de datos, seguridad y costo de una topología multi-región	Unidad D
RA6	Elaborar y justificar un diseño preliminar con un ADR	Taller

Agenda: apertura 8 min · A 27 min · B 40 min · pausa 10 min · C 40 min · D 15 min · taller 30 min · cierre 10 min.
Distribución ajustada a 60 % teoría, 16 % demostraciones y 24 % taller y discusión, porque el tema incluye un ejercicio de diseño.

De la Sesión 1 a la Sesión 2: de componentes a topologías

Sesión 1 · qué componentes existen

- Arquitectura de referencia (D-06)
- Prefill, decode, KV cache, batching
- Gestionado vs. autogestionado
- Mapa de servicios en tres nubes
- OWASP LLM 2025 por capa
- TTFT, TPOT y costo por token

Sesión 2 · dónde y cuántas copias

- Qué carga es online y cuál es batch
- Cuántas zonas y regiones, y con qué estrategia
- Quién conmuta: el proveedor, un gateway o el cliente
- Qué servicio implementa cada carga
- Qué cuesta la redundancia y dónde viajan los datos
- Diseño preliminar con ADR

Topología = componentes + ubicación + réplicas + rutas de tráfico + estrategia de falla

Unidad A · Inferencia online vs. por lotes

¿Qué cambia en la arquitectura cuando nadie está esperando la respuesta?

Un espectro de patrones de inferencia

Síncrono	Streaming	Asíncrono	Micro-batch	Batch
Solicitud → respuesta completa HTTP request/response	Tokens a medida que se generan SSE o WebSocket	Encolar → procesar → notificar cola + workers	Agrupar cada N s o N ítems flujos casi en tiempo real	Archivo de miles de solicitudes ventana de horas



Usuario esperando · SLO de latencia (TTFT, p95) — **Nadie esperando · SLO de plazo y costo por ítem**

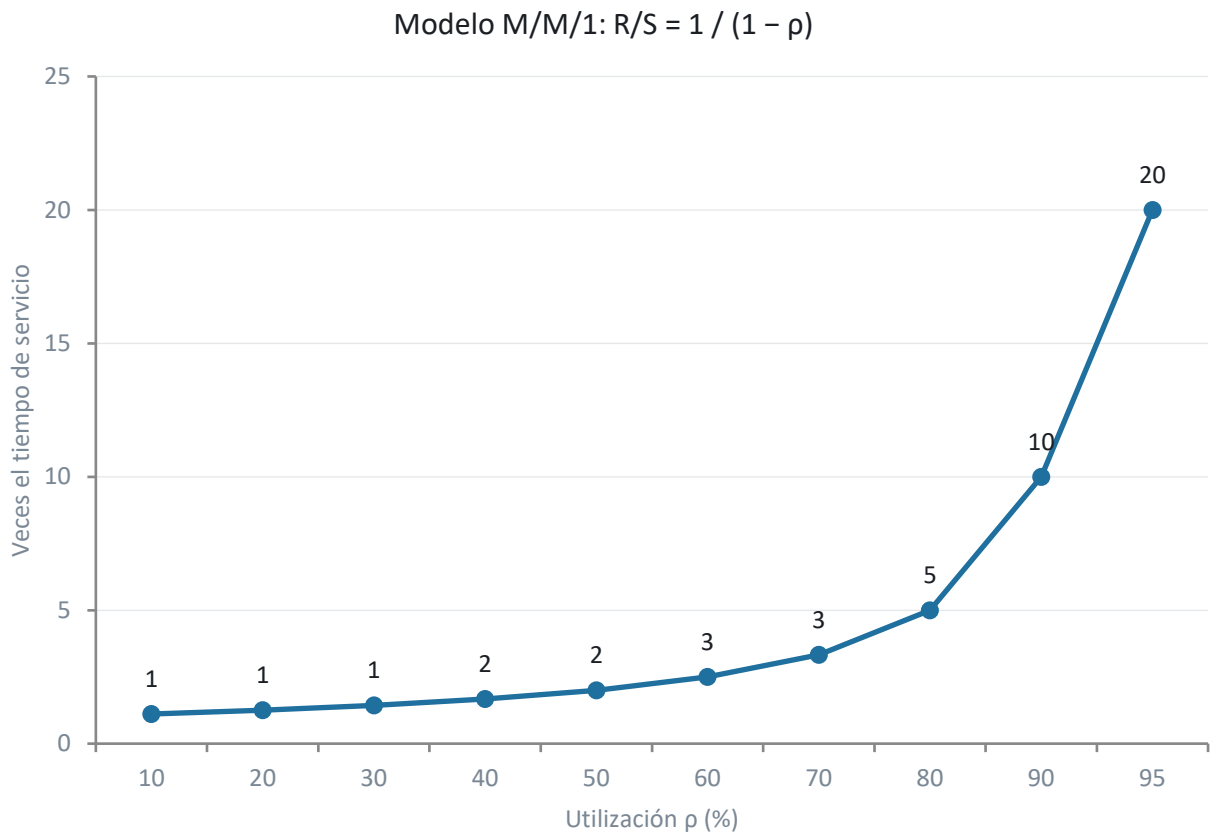
- Online (síncrono y streaming): el usuario está en la línea; cada segundo de espera se percibe
- Asíncrono: el usuario no espera en la conexión, pero sí espera el resultado (minutos)
- Batch: el resultado se consume después, en bloque (horas); el plazo es parte del contrato del servicio

El SLO define la carga de trabajo

Dimensión	Online (síncrono / streaming)	Por lotes (batch)
Quién espera	Una persona, en la conexión	Un proceso o un equipo, mañana
SLI principal	TTFT y latencia p95; tasa de errores	Tiempo hasta completar el lote; % de ítems exitosos
Unidad de costo	Costo por conversación o por 1.000 tokens	Costo por 1.000 ítems procesados
Capacidad	Holgura para picos; utilización moderada	Utilización alta; la cola es aceptable
Falla de un ítem	Error visible de inmediato	Se reintenta; el lote puede terminar parcial
Orden de resultados	Uno a uno	No garantizado: se empareja por identificador
Ejemplo del caso	Asistente de estudiantes	Clasificación nocturna de PQRS

Pregunta de diseño: ¿cuál es el costo de responder en 6 horas en vez de 6 segundos?

Por qué el lote es más barato: utilización y cola



- La espera en cola crece de forma no lineal con la utilización
- Online: para cumplir un p95 de latencia se opera con holgura (capacidad ociosa = costo)
- Batch: la espera es aceptable; el proveedor llena huecos y opera con utilización alta
- Por eso los servicios batch documentan descuentos del 50 % (Azure, Google Cloud, AWS) a cambio de una ventana de 24 h

M/M/1 es una simplificación (llegadas de Poisson, un servidor). Un servidor LLM con continuous batching se comporta distinto, pero la forma de la curva explica la intuición. Ver Harchol-Balter (2013).

Patrones de arquitectura para lotes

Batch gestionado

Archivo JSONL en almacenamiento → trabajo del proveedor → archivo de resultados

Ejemplos

Azure OpenAI Batch · Gemini batch inference · Bedrock batch

Trade-off

Mínimo esfuerzo; ventana de 24 h; límites de tamaño y de registros

Cola + workers propios

Mensajes en una cola → workers con concurrencia limitada → resultados en BD

Ejemplos

Service Bus · Pub/Sub · SQS + contenedores o funciones

Trade-off

Control fino del ritmo y la prioridad; más operación

Micro-batching

Agrupar solicitudes cercanas en el tiempo y enviarlas juntas

Ejemplos

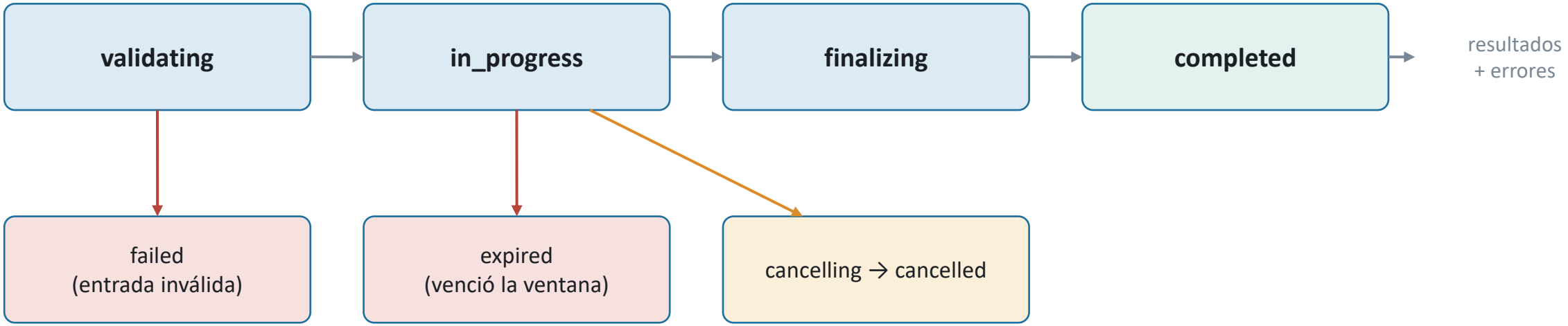
Ingesta de eventos, embeddings de documentos nuevos

Trade-off

Latencia de segundos; mejor uso que uno a uno

Regla práctica: use el batch gestionado salvo que necesite controlar el ritmo, la prioridad o un plazo menor que la ventana

Ciclo de vida común de un trabajo batch



Estado (Azure)	Gemini (google-genai)	Bedrock
validating · in_progress	JOB_STATE_PENDING · RUNNING	Submitted · Validating · Scheduled · InProgress
completed	JOB_STATE_SUCCEEDED · PARTIALLY_SUCCEEDED	Completed · PartiallyCompleted
failed · expired · cancelled	JOB_STATE_FAILED · EXPIRED · CANCELLED	Failed · Expired · Stopped

Batch gestionado en las tres nubes (20-oct-2026)

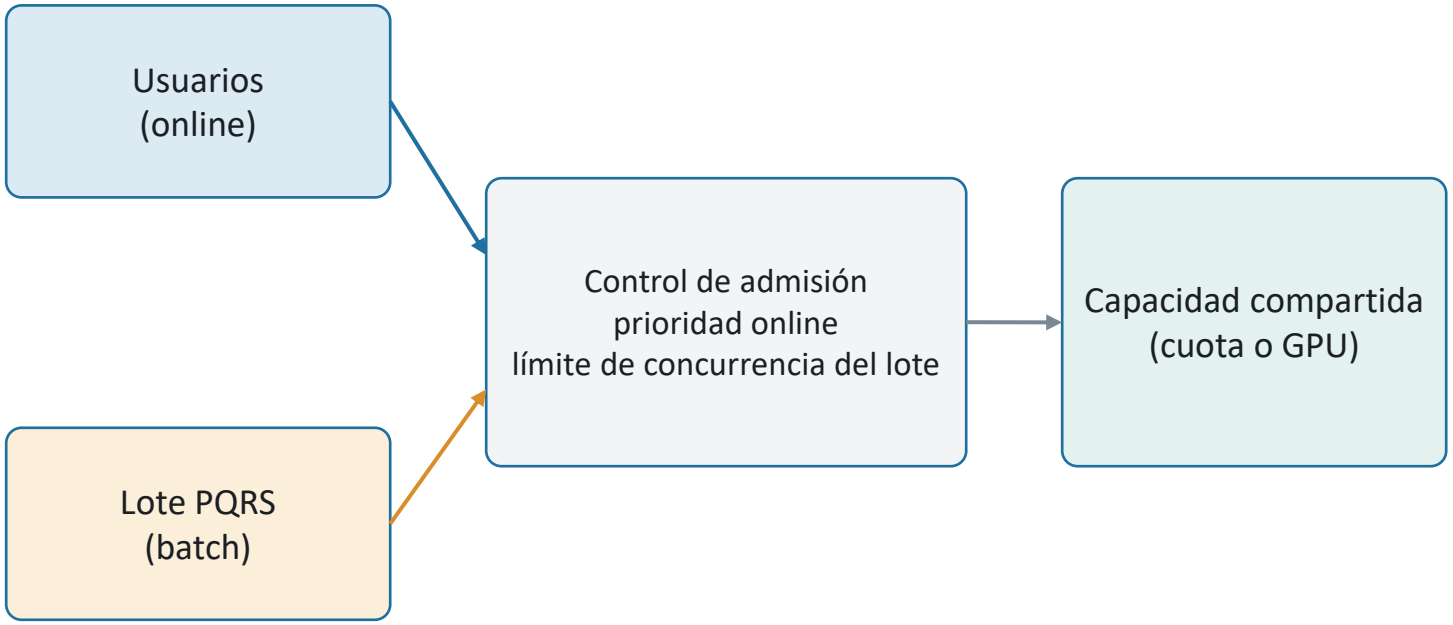
	Azure OpenAI Batch	Gemini batch inference	Amazon Bedrock batch
Entrada	JSONL (custom_id, method, url, body); hasta 100.000 solicitudes y 200 MB por archivo	JSONL en Cloud Storage o tabla de BigQuery	JSONL en S3 (recordId, modelInput); invocación InvokeModel o Converse
Ventana	24 h (objetivo; no expira automáticamente)	Según el trabajo; sin SLA de latencia	24 h
Precio documentado	50 % menos que Global Standard	50 % de descuento en modo batch	50 % de ahorro frente a on-demand
Mínimos y cuota	Cuota de tokens encolados separada de la online	Cuotas del proyecto	Mínimo 100 registros por trabajo
Residencia	Global Batch o Data Zone Batch	Según la ubicación del trabajo	Región única o perfil entre regiones
Cuenta trial	No disponible (cuota «N/A»)	Disponible con el crédito de prueba (validar)	Plan Paid recomendado

Clases de servicio online: prioridad, estándar y flexible

	Microsoft Foundry	Gemini Enterprise Agent Platform	Amazon Bedrock
Prioritaria	Priority processing (por despliegue o por solicitud con service_tier); Global y Data Zone Standard US	Priority PayGo	Priority (service_tier = priority)
Estándar	Standard (pago por token)	Standard PayGo	Standard (default)
Flexible	Flex processing para cargas tolerantes a demora	Flex PayGo	Flex (service_tier = flex)
Reservada	Provisioned (PTU)	Provisioned Throughput	Reserved (1 o 3 meses)

- La clase de servicio se elige por solicitud: el mismo endpoint atiende cargas con SLO distintos
- Flex es un punto intermedio entre online y batch: sin archivo ni ventana, pero puede tardar o limitarse
- Priority no aumenta la cuota: en Azure y en Bedrock comparte la cuota on-demand

Coexistencia: que el lote no degrade lo online



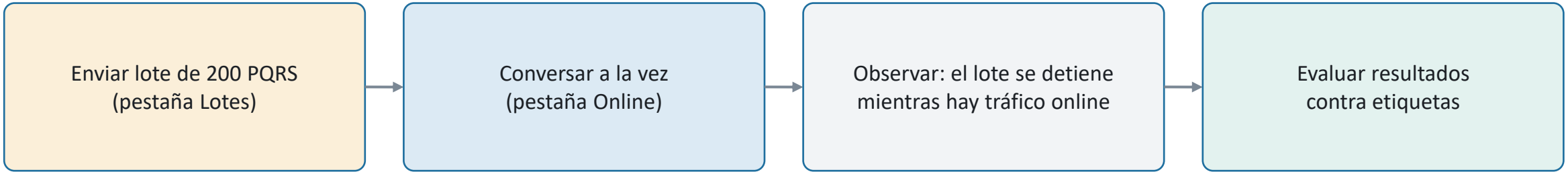
- Separar cuotas: Azure Batch usa una cuota de tokens encolados distinta de la online
- Separar endpoints o despliegues por carga cuando la cuota es compartida
- Prioridad y ritmo: el lote cede capacidad cuando hay tráfico interactivo (LOC-02)
- Programar en horas valle: el lote nocturno aprovecha la capacidad ociosa
- Observar por separado: SLI online y SLI batch en tableros distintos

ADR-0006: separar los caminos online y batch, con prioridad explícita para lo interactivo

Riesgos propios de la inferencia por lotes

Riesgo	Qué pasa	Control
Idempotencia	Un reintento procesa dos veces el mismo ítem	Identificador por ítem (custom_id, recordId); reanudar sin repetir lo resuelto
Resultados parciales	El lote vence o se cancela a mitad	Tratar completed, expired y cancelled como estados normales; reprocesar faltantes
Calidad silenciosa	Miles de respuestas mal formadas sin que nadie lo note	Validar el formato de salida y medir exactitud contra una muestra etiquetada
Inyección indirecta (LLM01)	Un ítem contiene instrucciones para el modelo	Tratar el contenido como dato; validar la salida contra un esquema
Consumo sin límite (LLM10)	Un archivo mal generado multiplica el costo	Límites de registros y tamaño; presupuesto y alertas
Datos en reposo (LLM02)	Entradas y salidas quedan en almacenamiento	Cifrado, caducidad (Azure: 14 a 30 días), acceso mínimo

Demo 1 · Online y lote sobre la misma capacidad (LOC-02)



- Antes de ejecutar, prediga: ¿subirá el TTFT de la conversación mientras corre el lote?
- Observe en la tabla de lotes el progreso y las regiones usadas
- Descargue los resultados y ejecute evaluar_lote.py: JSON válidos, exactitud por tipo y por área
- Mismo archivo, mismo formato que Azure OpenAI Batch: la diferencia es quién opera la cola

Plan B: tabla de resultados sintéticos del README de LOC-02 (sección 9)

Unidad B · Topologías redundantes y multi-región

¿Cuántas copias necesita el sistema y quién decide a cuál enviar el tráfico?

Vocabulario de continuidad

Disponibilidad	Proporción del tiempo en que el servicio cumple su función. 99,9 % ≈ 43 min de indisponibilidad al mes
Dominio de falla	Conjunto de recursos que fallan juntos: un host, una zona, una región, un proveedor, una cuota
RTO	Recovery Time Objective: máximo tiempo aceptable entre la interrupción y la restauración
RPO	Recovery Point Objective: máxima pérdida de datos aceptable, medida en tiempo
Radio de impacto	Cuántos usuarios o funciones afecta una falla (blast radius)
Resiliencia	Capacidad de resistir la falla y recuperarse: detectar, aislar, conmutar, degradar

Definiciones de RTO y RPO según AWS Architecture Blog, «Disaster Recovery (DR) Architecture on AWS, Part I» (Eliot, 2021).

Disponibilidad compuesta: serie y paralelo

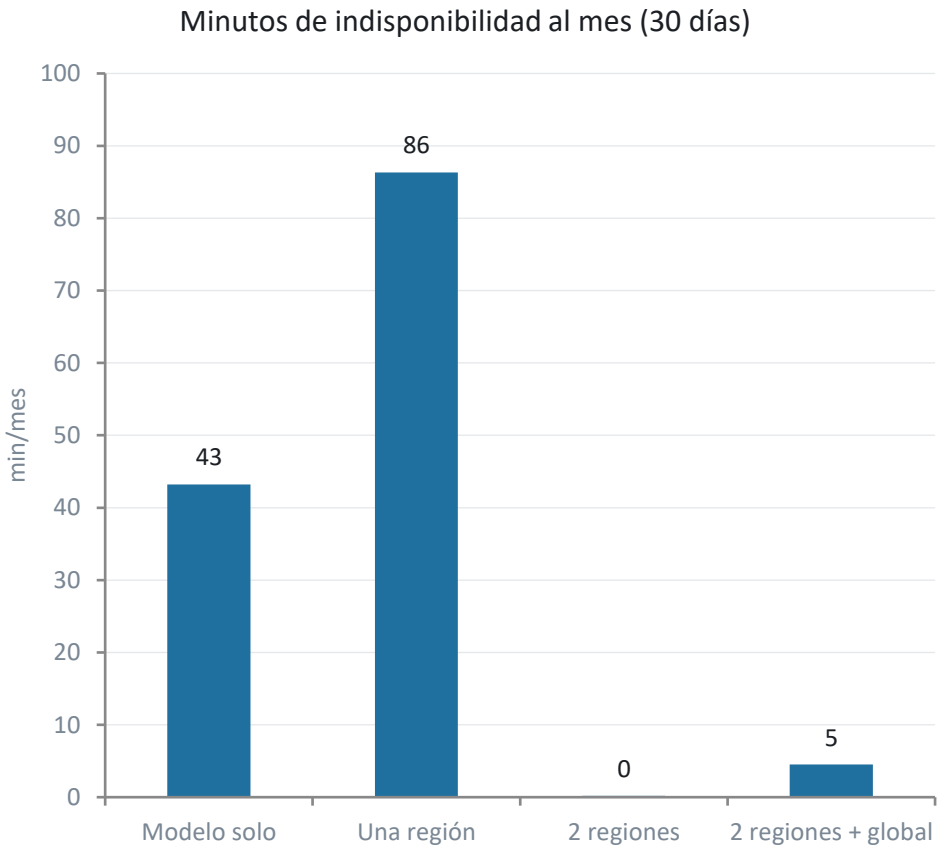
En serie (todos deben funcionar)

$$A = A_1 \times A_2 \times \ldots \times A_n$$

En paralelo (basta con uno, fallas independientes)

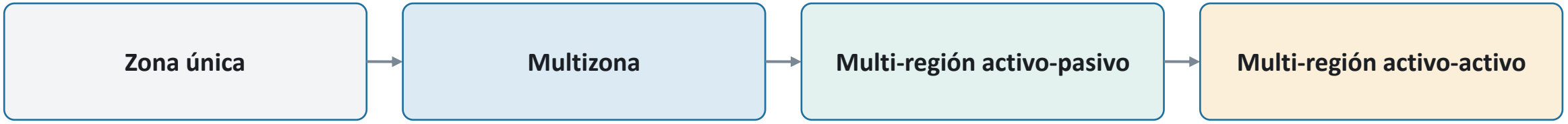
$$A = 1 - (1 - A_1) \times (1 - A_2)$$

Configuración (cifras hipotéticas)	A	Indisp./mes
Solo el endpoint del modelo (99,9 %)	99,90 %	43 min
Una región: gateway 99,95 × backend 99,95 × modelo 99,9	99,80 %	86 min
Dos regiones en paralelo	99,9996 %	0,2 min
Dos regiones + enrutamiento global en serie (99,99 %)	99,99 %	4,5 min



Cada componente en serie resta; el componente global en serie se vuelve el techo

Espectro de topologías



Zona única	Multizona	Multi-región activo-pasivo	Multi-región activo-activo
1 zona · 1 región	≥ 2 zonas · 1 región	Primaria + secundaria en espera	≥ 2 regiones con tráfico
Fallas de host; no de zona	Falla de zona; no de región	Falla de región, con RTO de minutos a horas	Falla de región con RTO cercano a cero
$	$$	$$–$$$	$$$

- Multizona es la línea base razonable para producción; multi-región se justifica por RTO, regulación o riesgo del negocio
- Activo-activo exige que los datos con estado se repliquen y que los clientes toleren respuestas desde cualquier región

Estrategias de recuperación ante desastres

Estrategia (AWS)	Equivalente (Microsoft)	En la región secundaria	RTO	RPO	Costo
Backup and restore	Hot/cold	Solo copias de datos	Horas	Horas	Bajo
Pilot light	—	Datos replicados; cómputo apagado	Decenas de minutos	Minutos	Bajo-medio
Warm standby	Hot/warm	Despliegue mínimo funcionando	Minutos	Segundos-minutos	Medio-alto
Multi-site active/active	Hot/hot	Despliegue completo con tráfico	Cercano a cero	Cercano a cero	Alto

- Microsoft da rangos aproximados para Foundry: hot/hot en minutos; hot/warm de 30 min a 2 h; hot/cold de 2 a 8 h
- En LLM gestionados, el «warm standby» es barato: un despliegue de pago por token sin tráfico no factura tokens
- En LLM autogestionados, la GPU en espera se paga por hora: el pilot light deja la GPU apagada y paga con arranque en frío

¿Qué hay que replicar en un sistema LLM?

Componente	Estado	Estrategia de réplica	RPO típico
Gateway, backend, workers	Sin estado	Desplegar en cada región desde el mismo artefacto (IaC + imagen)	No aplica
Configuración: prompts de sistema, guardrails, versión del modelo	Estado de control	Versionar en Git y desplegar a todas las regiones a la vez	Cero
Conversaciones y memoria de agentes	Estado de usuario	Base de datos con réplica entre regiones o aceptar pérdida	Segundos a horas
Documentos e índice vectorial (RAG)	Estado derivado	Replicar documentos; reconstruir o replicar el índice	Horas
Modelo gestionado	Capacidad del proveedor	Despliegue y cuota en cada región; misma versión	No aplica
Modelo autogestionado	Pesos + GPU	Registro de imágenes replicado; capacidad de GPU reservada o verificada	No aplica
Secretos e identidades	Estado de control	Gestor de secretos con réplica; identidades de carga por región	Cero

Particularidades de los LLM en multi-región

1

Disponibilidad del modelo por región

No todos los modelos ni versiones existen en todas las regiones. Verifique antes de elegir la secundaria

2

Cuota compartida

En Foundry, desde el 7-may-2026, la cuota de Global Standard es por suscripción y se comparte entre regiones

3

Misma versión del modelo

Conmutar a otra versión cambia las respuestas; la guía de gateways de Azure exige el mismo modelo y versión

4

Capacidad de GPU escasa

En autogestionado, la región secundaria puede no tener GPU disponible cuando se necesita

5

Arranque en frío

Cargar decenas de GB de pesos hace lenta la activación de una secundaria apagada

6

Conversaciones largas

Conmutar a mitad de una respuesta en streaming duplica o corta texto

Dos capas de multi-región: la del proveedor y la propia

Capa del proveedor (dentro del servicio de modelos)

Nube	Mecanismo	Alcance del procesamiento
Azure	Global / Data Zone Standard	Cualquier región / zona de datos
Google Cloud	Endpoint global / multirregional	Cualquier ubicación / jurisdicción
AWS	Cross-Region inference	Geografía (us., eu.…) o global

Capa propia (antes del servicio de modelos)

- Backend y gateway en dos regiones
- Conmutación entre endpoints regionales
- Datos con estado replicados
- Enrutamiento global de usuarios

- Microsoft: el enrutamiento de Global y Data Zone «no reemplaza el failover regional a nivel de aplicación»
- Google: el endpoint global no ofrece aislamiento regional ni garantías de residencia
- AWS: CRIS no cambia dónde se almacenan los datos; las SCP deben permitir todas las regiones de destino

La capa del proveedor mejora la capacidad; la capa propia protege su endpoint, sus datos y su aplicación

Enrutamiento global: ¿cómo llega el usuario a la región sana?

Mecanismo	Ejemplos	Detección y conmutación	Consideraciones
DNS con health checks	Route 53 failover · Traffic Manager · Cloud DNS routing policies	Health check + TTL del registro	Los clientes y resolvers guardan en caché: la conmutación tarda lo que el TTL o más
Balanceador global (anycast)	Azure Front Door · Cloud Load Balancing global · Global Accelerator	Segundos; la IP no cambia	Un punto de entrada global con WAF; es otro componente en serie
Gateway de IA	API Management · Apigee · gateway propio (LOC-02)	Por solicitud (circuit breaker)	Ve errores 429 y 5xx del modelo; puede ser punto único de falla
Cliente o SDK	Lógica de conmutación en la aplicación (AZ-G, GCP-D, AWS-E)	Por solicitud	Repite lógica en cada cliente; útil en prototipos

Combinación habitual: balanceador global para los usuarios + gateway por región para los modelos

Patrones de resiliencia para la ruta de inferencia

Timeouts	Límite de conexión y de primer byte; sin él, una región lenta retiene recursos
Reintentos con backoff y jitter	Esperas crecientes y aleatorias; respetar Retry-After en 429
Presupuesto de reintentos	Máximo de intentos por solicitud y por capa
Circuit breaker	Dejar de enviar tráfico a una región que falla y probar su recuperación
Bulkhead	Aislar capacidad por carga (online, batch) o por cliente
Degradación	Modelo más pequeño, respuesta en caché o mensaje claro de indisponibilidad
Load shedding	Rechazar con 429 o 503 lo que no se puede atender, antes de saturarse
Idempotencia	Repetir sin efectos duplicados (lotes, herramientas de agentes)

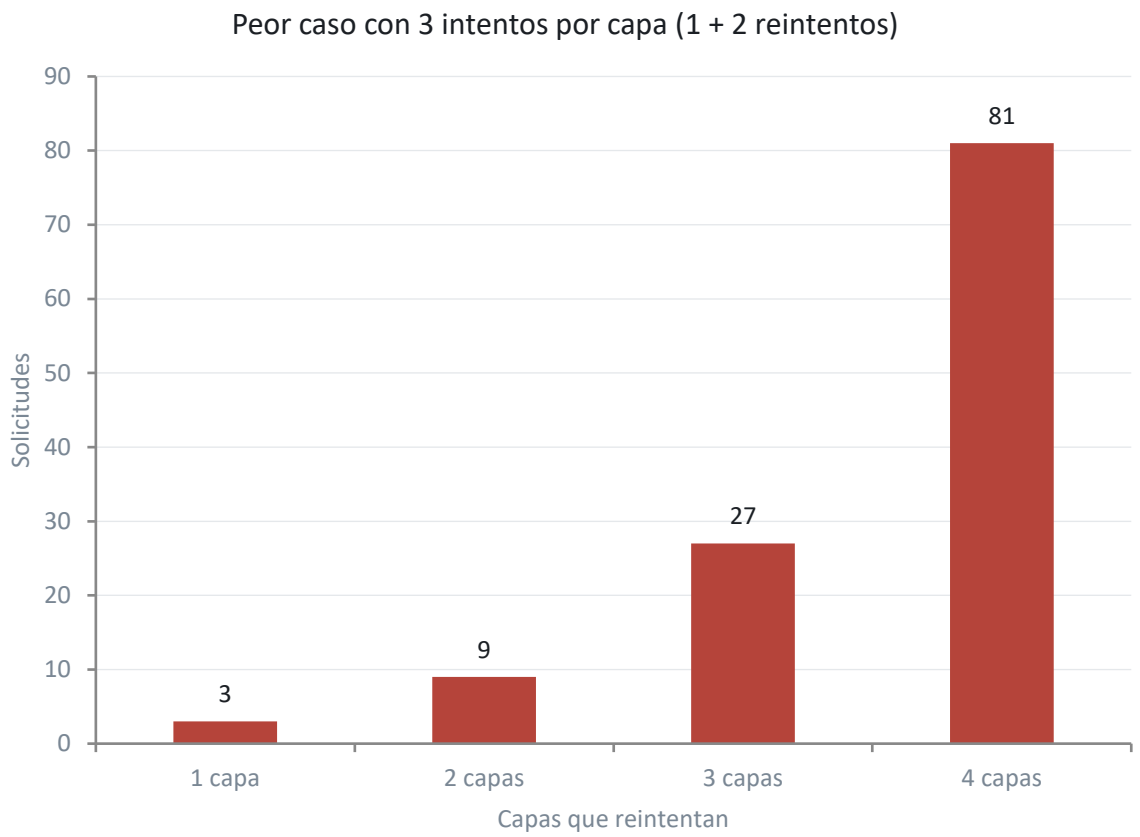
Referencias: Nygard, «Release It!» (2.ª ed., 2018); Beyer et al., «Site Reliability Engineering» (2016), cap. 21–22.

Circuit breaker: tres estados



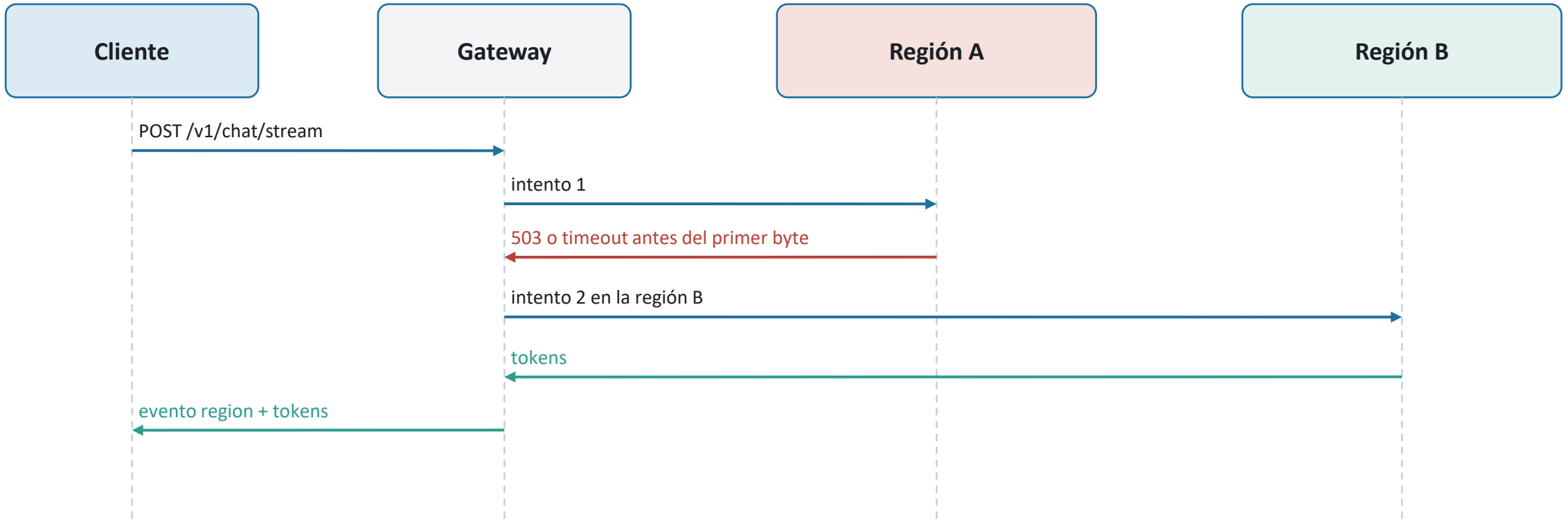
- Evita insistir sobre una región caída y da tiempo a que se recupere
- Un 429 con Retry-After abre el circuito exactamente ese tiempo (no adivinar el throttling)

Tormentas de reintentos: la amplificación



- Frontend, gateway, backend y SDK pueden reintentar cada uno por su cuenta
- Durante un incidente, la carga se multiplica justo cuando el proveedor está limitado
- Reintentar en una sola capa; las demás fallan rápido
- Desactivar los reintentos automáticos del SDK cuando un gateway conmuta (AZ-G, GCP-D y AWS-E lo hacen)
- Presupuesto global: por ejemplo, reintentos ≤ 10 % del tráfico

Streaming y conmutación: solo antes del primer token



- Si la región falla después del primer token, conmutar duplicaría o mezclaría texto: se informa el error
- El estado de la conversación debe estar fuera de la región para que la siguiente pregunta funcione en B

El gateway de IA: ventajas y riesgos

Lo que aporta

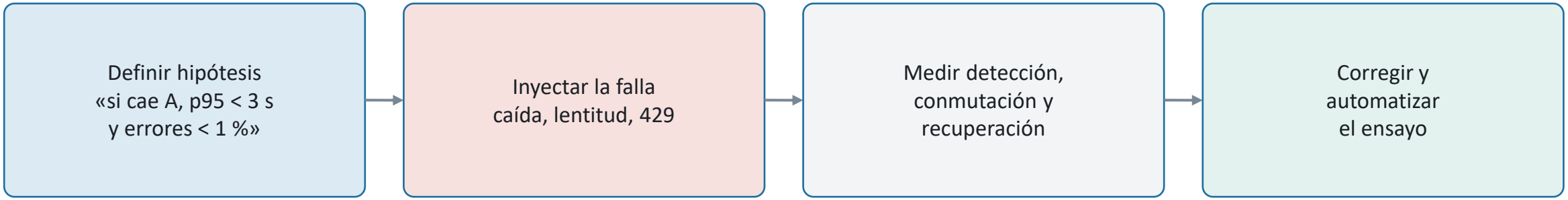
- Conmutación y circuit breaker en un solo lugar
- Límites de tokens por cliente (p. ej., llm-token-limit en APIM)
- Observabilidad central: tokens, latencia, región
- Prioridad entre cargas y entre despliegues (PTU primero, pago por uso después)
- Contrato propio: cambiar de proveedor sin tocar clientes

Lo que arriesga

- Nuevo componente en serie: su disponibilidad es el techo
- Si vive en una sola región, vuelve a ser punto único de falla
- Latencia adicional por salto
- Un gateway unificado entre jurisdicciones puede violar la residencia
- Más superficie de ataque y más operación

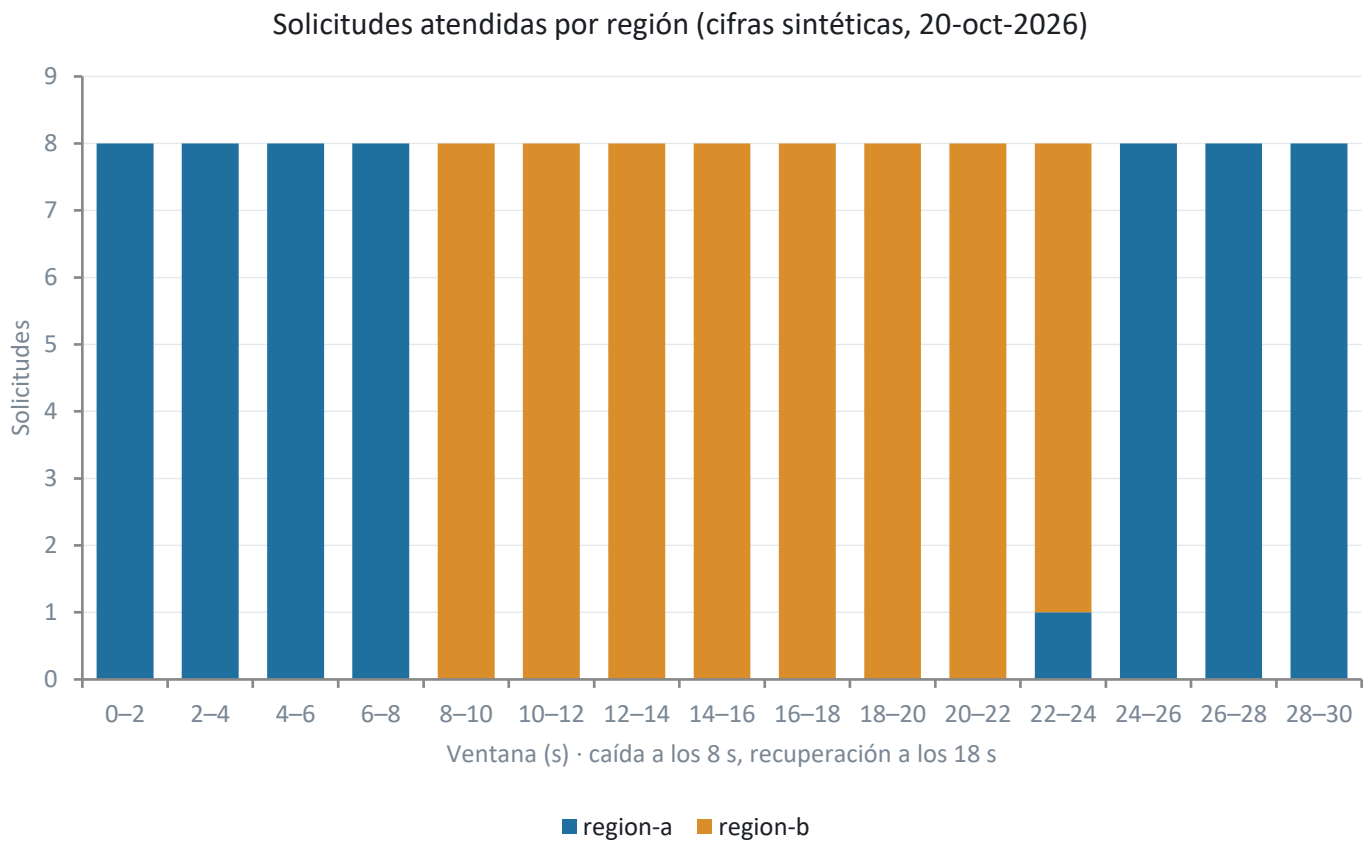
Microsoft: no implementar un gateway unificado entre regiones geopolíticas cuando aplica residencia de datos

Probar la conmutación antes del incidente



- Microsoft recomienda verificar periódicamente (por ejemplo, cada trimestre) despliegues, conexiones, RBAC, DNS y una prueba de punta a punta en la región secundaria
- Una secundaria que nunca recibió tráfico no está probada: cuota, permisos o versión del modelo pueden diferir
- Las pruebas se hacen primero en un entorno de laboratorio (LOC-02) y luego como game days controlados

Demo 2 · Falla regional bajo carga (LOC-02)



- 120 solicitudes, 0 errores, 3 conmutaciones
- La primaria vuelve 4–6 s después de recuperarse: circuito abierto (15 s) e intervalo del health check (5 s)
- Pregunta: ¿qué detectó primero la falla, el health check o el circuit breaker?

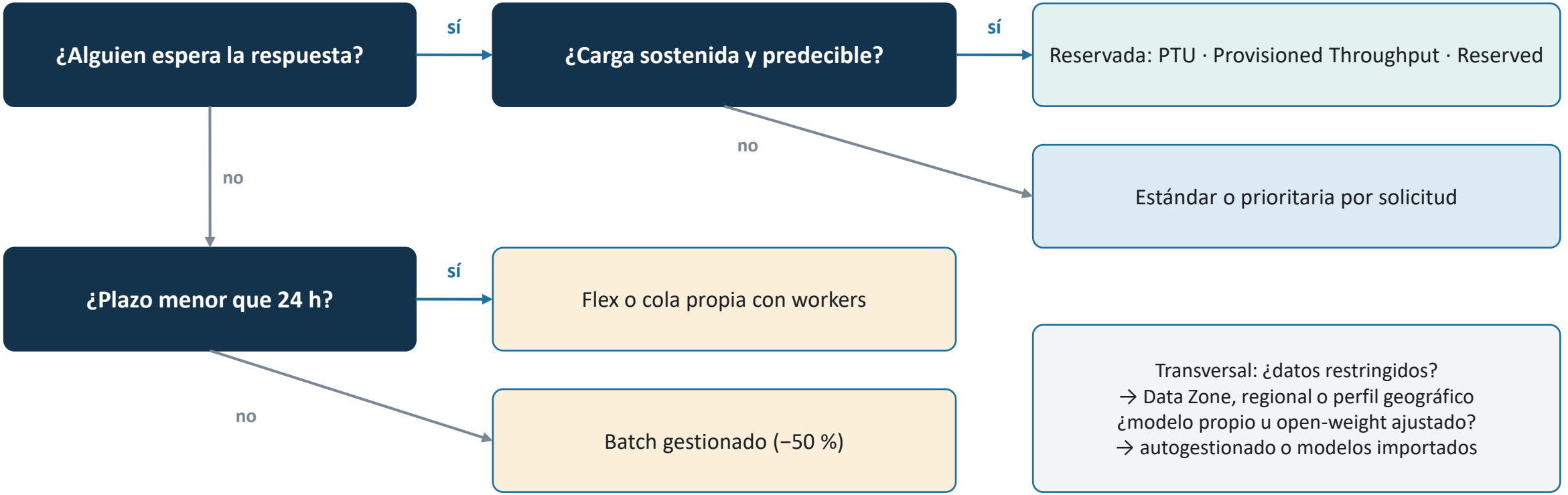
Unidad C · Selección de servicios gestionados según el caso de uso

¿Con qué criterios se elige el servicio, el tipo de despliegue y la modalidad de consumo?

Criterios de selección

Familia	Criterio	Pregunta de diseño
Funcional	Modelos y capacidades	¿Está el modelo (y la versión) que necesito, con herramientas, JSON estructurado y contexto suficiente?
Ubicación	Regiones y residencia	¿Dónde se procesa y dónde se almacena? ¿Hay una segunda región con el mismo modelo?
Rendimiento	Latencia, cuota y throughput	¿La cuota cubre el pico en tokens por minuto? ¿Qué latencia p95 obtengo?
Economía	Modalidad de consumo	¿Pago por token, prioridad, flex, batch, reservado o GPU propia?
Confiabilidad	SLA y madurez	¿Es GA o vista previa? ¿Qué SLA publica el proveedor para esa modalidad?
Seguridad	Red, identidad, cifrado	¿Endpoints privados, identidad sin secretos, claves propias, registro de auditoría?
Operación	Ecosistema y habilidades	¿Qué nube usa ya la institución y qué sabe operar el equipo?
Estrategia	Portabilidad	¿Qué tan costoso es cambiar? ¿Uso APIs y estándares compatibles?

Árbol de decisión de la modalidad de consumo



La guía de costos de Bedrock sugiere evaluar la capacidad reservada cuando la carga sostenida supera aproximadamente el 40 % del costo on-demand; calcule su propio punto de equilibrio.

Una misma aplicación suele combinar dos o tres modalidades

Mapa de servicios por caso de uso (20-oct-2026)

Caso de uso	Microsoft Azure	Google Cloud	AWS
Conversación online	Foundry · Global/Data Zone/Standard; Priority processing	Gemini en Agent Platform · global, multirregional o regional	Bedrock on-demand · CRIS · Priority
Lote nocturno	Azure OpenAI Batch (Global o Data Zone Batch)	Gemini batch inference (Cloud Storage, BigQuery)	Bedrock batch inference (S3)
Plazo intermedio	Flex processing · Service Bus + Container Apps	Flex PayGo · Pub/Sub + Cloud Run	Flex · SQS + Lambda o ECS
Carga sostenida alta	Provisioned (PTU) Global, Data Zone o regional	Provisioned Throughput	Reserved tier
Gateway de IA	API Management	Apigee o gateway en Cloud Run	API Gateway + Lambda o gateway en ECS
Enrutamiento global	Front Door · Traffic Manager	Cloud Load Balancing global · Cloud DNS	Route 53 · Global Accelerator · CloudFront
Modelo autogestionado	AKS · Container Apps con GPU	GKE · Cloud Run con GPU	EKS · ECS · SageMaker

Azure: tipo de despliegue = residencia + precio + SLA

	Standard (pago por token)	Provisioned (PTU)	Batch
Global	Inferencia en cualquier región; mayor cuota predeterminada	Capacidad reservada, latencia más estable	−50 % frente a Global Standard; objetivo 24 h
Data Zone (EE. UU., UE, APAC)	Inferencia dentro de la zona de datos	Reservada dentro de la zona	−50 %; dentro de la zona
Regional (Standard)	Inferencia dentro de la geografía elegida	Reservada en la región	—

- Los datos en reposo permanecen en la geografía designada en todos los tipos
- Con Global y Data Zone Standard, si la región primaria se interrumpe, se afecta todo el tráfico enrutado a ella
- Priority processing: atributo del despliegue o de la solicitud (Global Standard y Data Zone Standard US)
- Developer: sin SLA ni garantía de residencia; solo para evaluar modelos ajustados

Google Cloud: ubicación del endpoint y modalidad de consumo

Ubicación del endpoint	Dónde se procesa	Uso típico
Global	Cualquier ubicación de Google Cloud; sin garantías de residencia	Máxima disponibilidad cuando no hay restricción de datos
Multirregional (us, eu)	Dentro de la jurisdicción; eu excluye Reino Unido y Suiza	Residencia con más capacidad que una región
Regional (p. ej., us-central1)	Dentro de la jurisdicción de la región	Residencia estricta; menor disponibilidad

Modalidad	Para qué
Standard PayGo	Tráfico general
Priority PayGo	Cargas críticas con tráfico fluctuante
Flex PayGo	Cargas no críticas tolerantes a la latencia
Provisioned Throughput	Capacidad reservada
Batch inference	Lotes desde Cloud Storage o BigQuery, −50 %

Los modelos anteriores a Gemini 3 no se admiten en los endpoints multirregionales jurisdiccionales.
Gemini 2.5 tiene retiro anunciado: actualice los laboratorios de la Sesión 1.

AWS: perfiles de inferencia y niveles de servicio

Opción	Comportamiento	Consideraciones
Modelo en una región	Se procesa en la región de origen	Máximo control de residencia; capacidad de una región
Perfil geográfico (us., eu., jp., au.)	Bedrock elige una región dentro de la geografía	IAM y SCP deben permitir todas las regiones de destino
Perfil global (global.)	Cualquier región comercial	Mayor capacidad; sin restricción geográfica
Niveles Priority, Standard, Flex	Se elige por solicitud con service_tier	Comparten la cuota on-demand
Reserved	Capacidad de tokens por minuto reservada por 1 o 3 meses	Mínimos de 100.000 TPM de entrada y 10.000 de salida; desborda a Standard
Batch	S3 → trabajo → S3	Mínimo 100 registros; −50 %

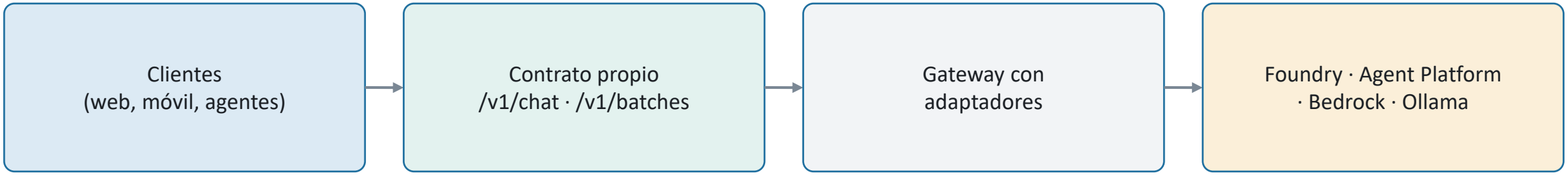
El plan Free de AWS no admite funciones multirregión ni entre regiones: para CRIS use el plan Paid con presupuesto

Gestionado vs. autogestionado en una topología multi-región

Aspecto	Modelo gestionado (API)	Modelo autogestionado (GPU propia)
Región secundaria en espera	Despliegue sin tráfico: sin cargo por tokens	GPU encendida = costo por hora; apagada = arranque en frío
Capacidad en el incidente	Cuota de la suscripción o proyecto	Disponibilidad de GPU en la región secundaria
Conmutación	Cambio de endpoint o capa del proveedor	Cambio de clúster; pesos e imágenes replicados
Versión del modelo	Decide el proveedor (retiros programados)	Decide usted (fija los pesos)
Residencia	Según tipo de despliegue o endpoint	Total: usted elige dónde corre

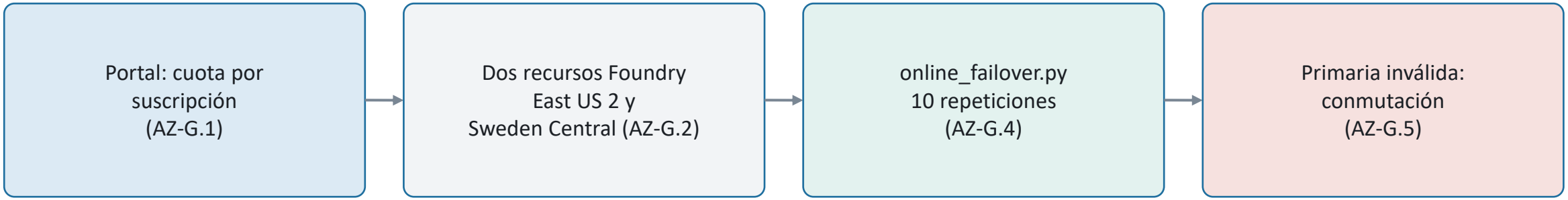
- Con el caso de la Sesión 1: 2 instancias L4 24/7 en Cloud Run ≈ US$1.507 al mes; el asistente con gpt-5-mini ≈ US$67,50 al mes
- La redundancia multi-región amplía esa diferencia: duplicar GPU duplica el costo fijo

Portabilidad: reducir el costo de cambiar



- Contrato de API propio (Sesión 1): los clientes no conocen al proveedor
- APIs compatibles con Chat Completions: el mismo gateway de LOC-02 habla con Ollama, con Foundry (/openai/v1) o con un simulador
- Formatos de lote equivalentes: un generador produce los tres formatos (lote_pqrs.py)
- OpenTelemetry con convenciones GenAI: la misma telemetría en cualquier nube
- El lock-in no desaparece: cuotas, identidad, red y datos siguen siendo propios de cada nube

Demo 3 · Azure: dos regiones y conmutación (AZ-G)



- Prediga: con la primaria inaccesible, ¿cuántos intentos verá en la primera y en la segunda solicitud?
- Observe: la segunda región responde, pero ambas comparten la cuota de Global Standard de la suscripción
- Si hay suscripción de pago: mostrar Azure OpenAI Batch con el mismo lote (AZ-G.6)

Plan B: grabación del docente + pruebas sin red del laboratorio (pytest)

Unidad D · Seguridad, residencia y costo de la redundancia

¿Qué se rompe en seguridad y en presupuesto cuando el sistema se vuelve multi-región?

Residencia de datos: qué viaja y qué se queda

Mecanismo	Inferencia (datos en uso)	Datos en reposo
Azure Global / Data Zone / Standard	Cualquier región / zona de datos / geografía	Geografía designada
Google endpoint global / multirregional / regional	Cualquier ubicación / jurisdicción / región	Ubicación del recurso
AWS perfil global / geográfico / región	Cualquier región comercial / geografía / región	Región de origen (logs, configuración)

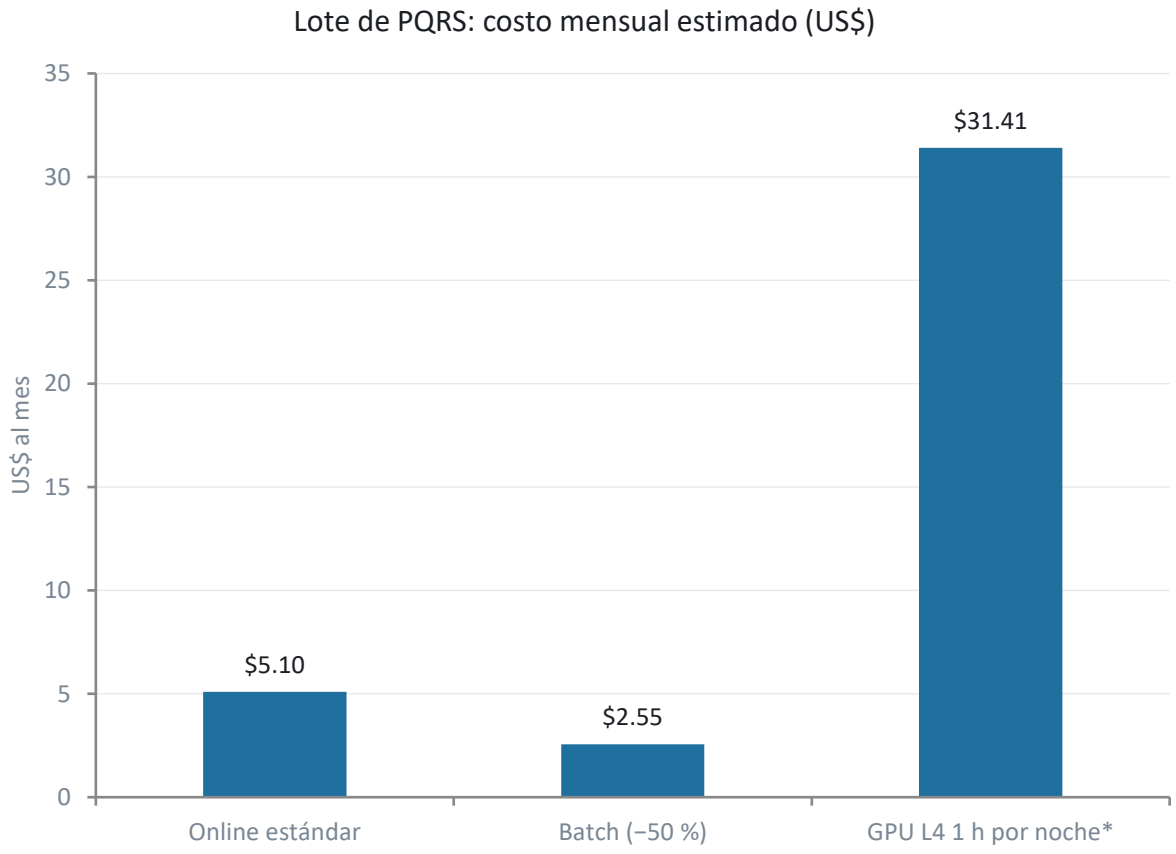
- Defina la clasificación de los datos antes de elegir el tipo de despliegue
- Las PQRS contienen datos personales: la base legal y las transferencias deben revisarse con la oficina jurídica (Ley 1581 de 2012)
- Documente en un ADR la combinación elegida y por qué es compatible con la política institucional

Seguridad en una topología multi-región

Identidad	Identidades de carga en cada región; mismos roles mínimos. Una SCP o política regional puede romper la conmutación
Red	Endpoints privados en cada región; la secundaria con los mismos controles que la primaria
Secretos	Gestor de secretos replicado; sin claves en archivos de configuración de regiones
Configuración	Prompts de sistema y guardrails idénticos en todas las regiones, desplegados desde Git
Registro y auditoría	Telemetría centralizada sin contenido de conversaciones; región y modalidad en cada traza
Lotes	Buckets cifrados y privados, rol de servicio con mínimo privilegio, caducidad de archivos

Principio: la región secundaria no puede ser la menos protegida (Zero Trust, NIST SP 800-207).

FinOps: ¿cuánto cuesta cada modalidad para el lote de PQRS?



- Supuestos: 20.000 PQRS al mes · 350 tokens de entrada y 60 de salida por PQRS → 7,0 M y 1,2 M tokens
- Precios publicados de Gemini 2.5 Flash (US$ por 1 M): 0,30 / 2,50 estándar y 0,15 / 1,25 batch, consultados el 20-oct-2026, usados solo para ilustrar la proporción
- * Cloud Run con GPU L4 a ≈ US$1,047 por hora (Sesión 1), 30 noches; supone que un modelo pequeño procesa el lote en 1 h (hipótesis no medida)

A este volumen, el batch gestionado es la opción más barata; la GPU solo compite con volúmenes mucho mayores

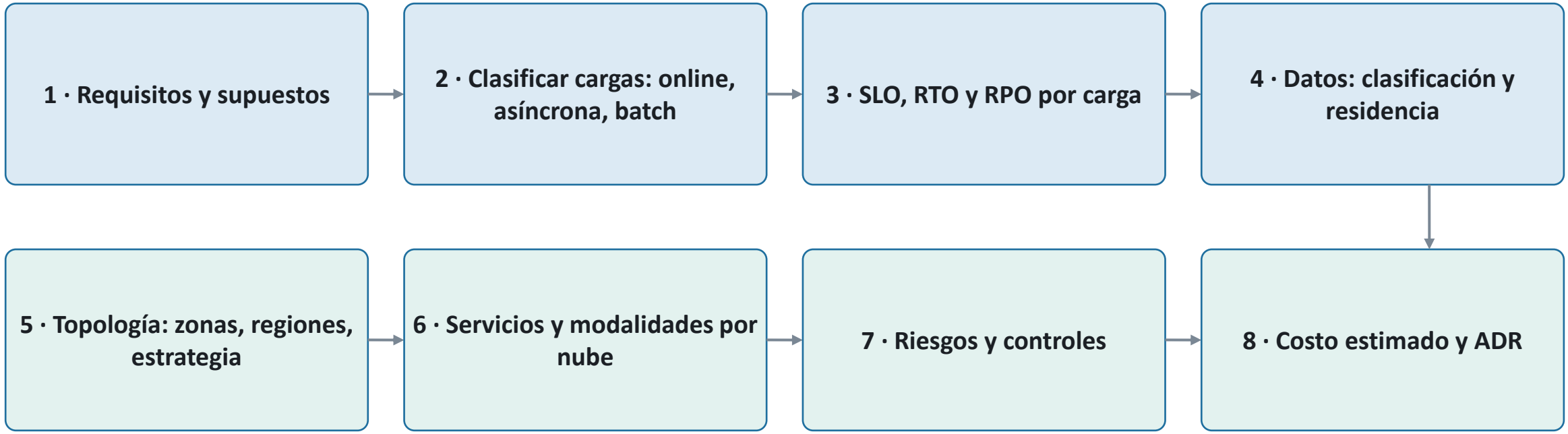
FinOps de la redundancia y de la topología

Fuente de costo	Comportamiento	Palanca
Región secundaria gestionada	Sin tokens, sin cargo por tokens	Mantenerla lista y probada; no reservar capacidad salvo necesidad
Región secundaria con GPU	Costo por hora aunque no reciba tráfico	Pilot light, escalado a cero o reserva compartida
Capacidad reservada	Pago fijo mensual	Dimensionar la base; desbordar a estándar
Gateway y balanceador global	Cargo por hora o por solicitud	Compartir entre aplicaciones; elegir nivel según funciones
Tráfico entre regiones	Transferencia de datos y replicación	Replicar solo lo necesario; índices reconstruibles
Priority	Precio superior al estándar	Solo para las solicitudes que lo necesitan
Batch y Flex	Descuento a cambio de plazo	Mover toda carga sin usuario esperando

Taller · Diseño preliminar de una topología

Aplicar la unidad completa a un caso empresarial y defender las decisiones

Método de diseño preliminar en ocho pasos



Un diseño preliminar no busca precisión: busca hacer explícitos los supuestos, las decisiones y sus trade-offs para poder validarlos.

Tres casos empresariales (elija uno por equipo)

A · Universidad

Asistente 24/7 para 5.000 estudiantes con picos en matrículas; 20.000 PQRS al mes clasificadas cada noche; reindexación semanal. Requisito: operar si cae una región; datos personales.

B · Retail regional

Chat de soporte en 4 países de Latinoamérica con 2.000 conversaciones concurrentes en temporada alta; generación nocturna de 50.000 descripciones de productos. Requisito: p95 < 3 s en el chat; presupuesto ajustado.

C · Entidad financiera

Análisis de solicitudes de crédito (documentos de 30 páginas) con respuesta en menos de 10 minutos; auditoría completa. Requisitos: inferencia solo en el país o la zona de datos; RTO 1 h; RPO 0 para las decisiones.

Las cifras son supuestos del ejercicio. Si falta un dato, regístrelo como supuesto y continúe.

Plantilla del diseño preliminar

Sección	Qué entregar
Cargas de trabajo	Lista de cargas con su patrón (online, asíncrona, batch) y su SLO
Objetivos de continuidad	Disponibilidad, RTO y RPO por carga, con la justificación
Datos	Clasificación y restricciones de residencia
Topología	Diagrama: regiones, zonas, enrutamiento global, gateway, datos replicados
Servicios	Servicio, tipo de despliegue o endpoint y modalidad de consumo, en una nube principal
Resiliencia	Patrones aplicados: timeouts, reintentos, circuit breaker, degradación
Riesgos	Tres riesgos principales con su control (incluido al menos uno de OWASP LLM)
Costo	Estimación por modalidad con supuestos explícitos
ADR	Una decisión con contexto, alternativas y consecuencias

Rúbrica y preguntas guía

Criterio	Peso
Clasificación de cargas y SLO coherentes	20 %
Topología justificada con RTO, RPO y dominios de falla	25 %
Selección de servicios y modalidades con criterios	20 %
Residencia, seguridad y riesgos	15 %
Costo con supuestos explícitos	10 %
ADR claro y defendible	10 %

- ¿Qué carga sobrevive a la caída de una región y cuál no? ¿Es aceptable?
- ¿Quién conmuta: el proveedor, el gateway o el cliente?
- ¿Qué componente global quedó en serie?
- ¿Dónde se procesa cada dato personal?
- ¿Qué pagaría aunque no hubiera tráfico?

Síntesis: seis ideas de la sesión

1. El SLO define la carga: latencia para lo online, plazo y costo por ítem para el lote
2. Lo que puede esperar, debe esperar: batch y flex cuestan menos
3. La disponibilidad compuesta se calcula; el componente global en serie es el techo
4. La multi-región del proveedor no reemplaza la propia: endpoint, datos y aplicación son suyos
5. Conmutar solo antes del primer token y con presupuesto de reintentos
6. Cada región copia también la superficie de ataque, la residencia y la factura

Autoevaluación (5 minutos)

#	Pregunta
1	Una aplicación clasifica 1 millón de correos al mes y los resultados se usan al día siguiente. ¿Qué modalidad elegiría y por qué?
2	Gateway 99,95 %, backend 99,95 % y modelo 99,9 % en serie, duplicados en dos regiones, más un DNS global de 99,99 %. ¿Cuál es la disponibilidad?
3	¿Por qué un segundo despliegue Global Standard en otra región de la misma suscripción no resuelve los 429 por cuota?
4	¿Qué ocurre si el SDK, el gateway y el backend reintentan cada uno 3 veces?
5	¿Qué tipo de despliegue o endpoint usaría si la inferencia debe ocurrir solo en EE. UU.? Responda para las tres nubes.

Laboratorios y próxima sesión

Laboratorio	Contenido	Cuenta trial
LOC-02	Gateway multi-región: failover, circuit breaker, API batch, React	No requiere nube
AZ-G	Dos regiones de Foundry, conmutación, Azure OpenAI Batch, APIM	Sí (batch: solo con suscripción de pago)
GCP-D	Ubicaciones de Gemini, conmutación, batch inference	Sí, con presupuesto
AWS-E	CRIS, niveles de servicio, conmutación, batch con rol de servicio	Plan Paid recomendado

- Tarea: completar el diseño preliminar del taller con su ADR y la estimación de costos
- Lectura: guía de gateways frente a múltiples despliegues (Azure Architecture Center) y estrategias de DR (AWS)
- Próxima sesión: sistemas distribuidos y contenedorización (Docker y Kubernetes) para servir LLMs

Referencias (1/2) · Documentación oficial

- Microsoft Learn. Deployment types for Microsoft Foundry Models (12-ago-2026); Getting started with Azure OpenAI batch deployments (5-jun-2026); Quotas and limits (20-ago-2026); Priority processing (22-sep-2026)
- Microsoft. High availability and resiliency for Microsoft Foundry (22-sep-2026); Azure Architecture Center, gateway frente a múltiples despliegues (2026); API Management backends (20-may-2026)
- Google Cloud. Batch inference with Gemini; Where your data lives and is processed; Priority PayGo y Flex PayGo; Pricing; notas de versión de Gemini Enterprise Agent Platform
- AWS. Service tiers for optimizing performance and cost; Capacity, limits and cost optimization; batch inference (formato, rol de servicio, modelos); Securing cross-Region inference (13-ene-2026)
- AWS Architecture Blog. Eliot, S. (2021). Disaster Recovery (DR) Architecture on AWS, Part I

Referencias (2/2) · Libros, estándares y artículos

- Beyer, B., Jones, C., Petoff, J. y Murphy, N. R. (2016). Site Reliability Engineering. O'Reilly
- Nygard, M. T. (2018). Release It! Design and Deploy Production-Ready Software (2.ª ed.). Pragmatic Bookshelf
- Harchol-Balter, M. (2013). Performance Modeling and Design of Computer Systems: Queueing Theory in Action. Cambridge University Press
- Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly
- OWASP. Top 10 for LLM Applications 2025 · NIST SP 800-207 (2020) · ISO/IEC 42001:2023 · Ley 1581 de 2012
- Agrawal et al. (2024). Sarathi-Serve, OSDI · Zhong et al. (2024). DistServe, OSDI